How predictable is tomorrow's Bitcoin volatility?

GARCH, realized variance and an honest 2025-2026 forecast test

Ignacio Queipo de Llano Pérez-Gascón
Predictive Statistics in Finance, IESE Master in Finance

October 2026

Summary. Bitcoin's direction is not predictable, but its risk is, partly. Volatility clusters strongly, and a Student-t GARCH describes it far better than a Normal one. Out of sample, over the 608 days from January 2025 to August 2026, the most important choice is not GARCH versus a simple regression on realized variance, but how much history the model uses. Bitcoin's volatility today is a fraction of what it was in 2017-2018, so models fitted on all history forecast risk 30-80% too high. Re-estimated on the last two years, the simple realized-variance regression is the best-calibrated and most accurate rule. Even so, it explains under a fifth of the day-to-day variation in risk.

1 Data, timing and how forecasts are judged

Daily returns are $r_t = 100\,[\log P_t - \log P_{t-1}]$ from 18 August 2017 to 31 August 2026: 3,301 calendar days, since Bitcoin trades every day. Models are estimated on data up to 31 December 2024 (2,693 days), and every forecast for day $t+1$ uses only information up to day t. The evaluation period is 1 January 2025 to 31 August 2026: 608 one-day-ahead forecasts.

What GARCH forecasts, $\sigma^2_{t+1|t} = \mathrm{Var}(r_{t+1} \mid \mathcal{F}_t)$, is never observed. I judge it against realized variance, $RV_{t+1} = \sum_{j=1}^{24} r^2_{t+1,j}$, the sum of the 24 squared hourly returns of that day. RV is far more informative than one squared daily return, but it is still a noisy proxy. All 608 evaluation days and their origins have a valid 24-hour RV. Two losses are used. MSE $= \frac{1}{T}\sum(RV_t - \hat{h}_t)^2$ punishes large errors in absolute terms, so a few spike days dominate it. QLIKE $= \frac{1}{T}\sum(\log \hat{h}_t + RV_t/\hat{h}_t)$ judges errors relative to the forecast and punishes under-prediction more than over-prediction. Both rank forecasts correctly even when RV is a noisy proxy (Patton, 2011), which is why the case uses them. Unless stated otherwise, standard errors are robust: QMLE for GARCH, Newey-West with 7 lags (one week) for regressions.

2 Part I: is there volatility to forecast?

Mean daily return	0.11%	Skewness	−0.75
Standard deviation	3.71% (71% a year)	Excess kurtosis	12.5
Worst / best day	−46.5% / +22.5%	Days beyond 4 s.d. (Normal: 0.17)	12
Ljung-Box, r_t, 10 / 20 lags	$p = 0.031$ / 0.076	Ljung-Box, r_t^2, 10 / 20 lags	$p < 10^{-15}$
Largest $\lvert\mathrm{ACF}(r_t)\rvert$, lags 1-20	0.045	$\mathrm{ACF}(r_t^2)$ at lag 1	0.099
ARCH-LM, 1 / 5 / 10 lags	$p = 4\times10^{-7}$ / 5×10^{-9} / 7×10^{-12}		

Table 1: Training sample, 18 Aug 2017 to 31 Dec 2024 (2,693 days). The 95% band for one autocorrelation is ±0.038.

(a) Returns are not meaningfully predictable from their own past. No return autocorrelation exceeds 0.045 in absolute value, which means yesterday's return explains at most 0.2% of today's. The Ljung-Box test rejects at 10 lags at the 5% level but not at 20. With 2,693 observations that is weak evidence of a tiny effect, not a usable signal.

(b) Volatility clusters. Squared returns are positively autocorrelated, and the Ljung-Box statistic for r_t^2 is five times that for r_t. Large moves, of either sign, tend to follow large moves. The distribution is also far from Normal: twelve days beyond four standard deviations where a Normal would give 0.17, led by −46.5% on 12 March 2020.

(c) What the ARCH-LM test does and does not establish. Rejecting the null hypothesis that squared residuals are uncorrelated establishes that the conditional variance is not constant. It does not establish that a GARCH model is the right description, how large or persistent the effect is, or that forecasts will be accurate. A change in the level of volatility between periods also produces autocorrelated squared returns without day-to-day

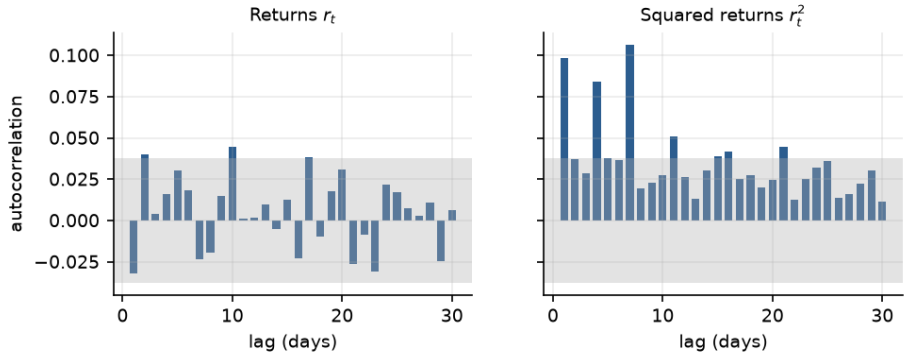


Figure 1: Autocorrelations in the training sample. Returns stay inside the noise band; squared returns stay above it.

clustering, and Bitcoin had such changes (Part VI). The test also says nothing about the mean: predictable risk does not mean predictable returns.

3 Part II and III: what GARCH learns, and why the distribution matters

	ARCH(1) Normal	GARCH(1,1) Normal	GARCH(1,1) Student-t
ω	11.90 (9.3)	0.66 (2.4)	0.13 (1.5)
α: reaction to yesterday's shock	0.136 (3.0)	0.100 (2.4)	0.073 (6.5)
β: weight on yesterday's variance	–	0.859 (19.2)	0.927 (61.2)
$\alpha + \beta$ [95% CI]	0.136	0.958 [0.925, 0.992]	1.000 [0.984, 1.016]
Half-life of a volatility shock	0.3 days	16 days	no decay
ν (degrees of freedom)	–	–	3.18 (20.1)
Log-likelihood	−7,307	−7,165	−6,831
AIC / BIC	14,620 / 14,638	14,337 / 14,361	13,673 / 13,702

Table 2: Estimates on the training sample, constant mean, robust t-statistics in brackets. The interval for $\alpha + \beta$ uses the joint covariance: $\hat{\alpha}$ and $\hat{\beta}$ are strongly negatively correlated (-0.92 and -0.85).

Why GARCH helps. ARCH(1) lets only yesterday's shock move today's variance, so a shock is forgotten within a day (half-life 0.3 days). This contradicts Figure 1, where squared returns stay correlated for weeks. Adding yesterday's variance (β) gives the model memory: GARCH(1,1) improves the log-likelihood by 142 points for one extra parameter, and both AIC and BIC prefer it clearly.

Reaction and persistence. In the Normal GARCH, a 10% daily move adds about $0.10 \times 10^2 \approx 10$ to tomorrow's variance (in $\%^2$). Starting from a calm 2.5% daily volatility, tomorrow's volatility jumps to about 4%. The excess then decays at rate $\alpha + \beta = 0.958$ a day: half of it is still there after 16 days. In the Student-t GARCH the immediate reaction is smaller ($\alpha = 0.073$), but the estimated persistence reaches the boundary of 1: shocks never fade and there is no long-run variance to return to. This is an integrated GARCH, equivalent to an exponentially weighted moving average of squared returns. Part VI suggests why it appears.

Significance is not forecast accuracy. α and β are clearly significant, but a significant t-statistic only says the parameter is not zero *inside the sample, if the model is right.* Forecast accuracy depends on other things too. The parameters must stay stable after the sample ends. The model's implied variance level must match the future level. And RV must not be so noisy that the gains disappear. With 2,693 days even modest effects are significant. Here α and β are individually blurred by their correlation, even though their sum is estimated precisely.

Normal versus Student-t. The t specification improves the log-likelihood by 333 points for one extra parameter (ν). $\hat{\nu} = 3.18$ (s.e. 0.16) means very heavy tails: with $\nu \leq 4$ the fourth moment does not exist, so kurtosis is infinite. Volatility clustering does not explain the tails away. The standardized residuals of the Normal GARCH still have excess kurtosis of 13.5. It helps to keep two choices apart. The *variance recursion* $\sigma_t^2 = \omega + \alpha\varepsilon_{t-1}^2 + \beta\sigma_{t-1}^2$ is the same in both models; only the *distribution of* z_t changes. But that choice still changes the recursion's estimates. Under a Normal, a 15% day is almost impossible, so the likelihood raises α and ω to make it less surprising. Under a t, the same day is an ordinary tail draw, so variance reacts less to each shock and persists more. For risk measures the distribution also matters directly: the same σ implies a much fatter left tail, which will matter for VaR and Expected Shortfall.

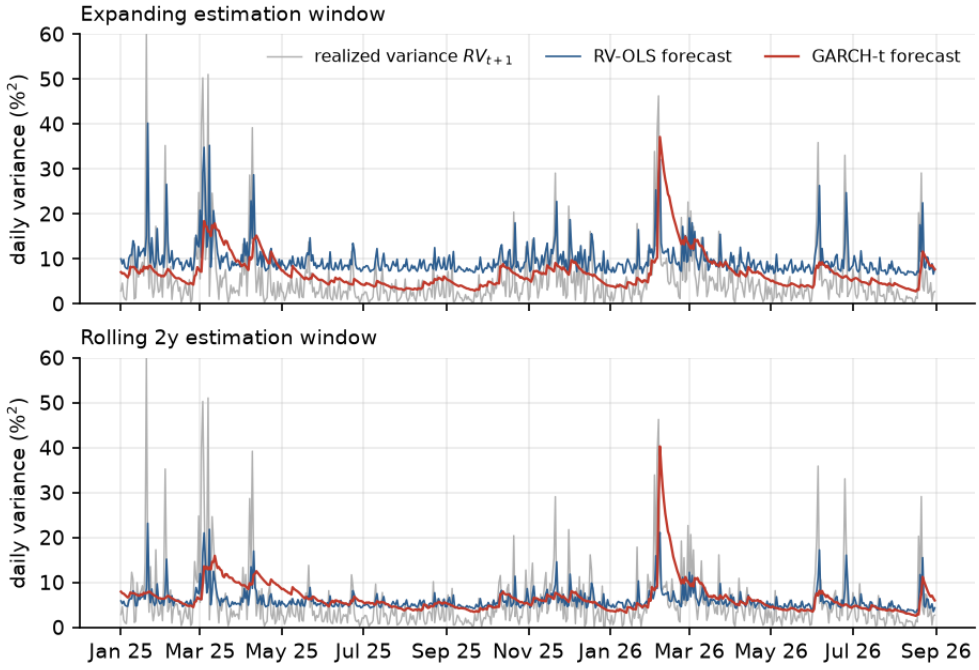


Figure 2: One-day-ahead variance forecasts against realized variance, 2025-2026. Spikes are cut at 60.

4 Part IV and V: forecasting a latent variable, and a simple benchmark

Each day from 1 January 2025 I re-estimate GARCH-t on all returns up to day t (expanding window) and forecast $\hat{h}_{t+1} = \hat{\sigma}^2_{t+1|t}$. The colleague's benchmark, *RV-OLS*, regresses tomorrow's RV on today's: $RV_{t+1} = a + b\,RV_t + u_{t+1}$, re-estimated in the same way on the same dates.[1]

The benchmark in sample. On the training data $\hat{b} = 0.548$ (Newey-West $t = 12.5$; plain OLS would claim 33), with 95% interval [0.46, 0.63] and $R^2 = 0.29$. Today's RV clearly carries significant information about tomorrow's. But $b = 0.55$ means a jump in RV loses half its effect within about a day: the forecast quickly returns to the regression's long-run mean, $\hat{a}/(1-\hat{b}) = 16.2$.

Rule	MSE	QLIKE	Corr($\hat{h}, RV$)	Mean $\hat{h}$	$\hat{h}$ / mean RV	MZ slope
GARCH-t, expanding	48.7	2.612	0.33	7.12	1.32	0.53
RV-OLS, expanding	58.1	2.753	**0.43**	9.76	1.82	0.77
GARCH-t, rolling 2 years	47.0	2.615	0.29	6.62	1.23	0.54
RV-OLS, rolling 2 years	**39.0**	**2.590**	0.42	5.97	**1.11**	1.22

Table 3: Out-of-sample scores, 608 days, 1 Jan 2025 to 31 Aug 2026. Mean realized variance: 5.38 (2.3% daily volatility). MZ slope: b in the Mincer-Zarnowitz regression $RV_t = a + b\,\hat{h}_t$; a well-scaled forecast has $b = 1$. The rolling rows belong to Part VI and are shown here for comparison.

What each statistic says. MSE measures accuracy in absolute terms and is driven by a handful of spike days: half-yearly MSE ranges from 16 to 86. QLIKE measures errors relative to the forecast's own level, so a forecast of 2 when RV turns out to be 10 costs much more than the reverse. Correlation measures only whether the forecast moves *with* RV. It ignores the level, so a forecast that is consistently 80% too high keeps its correlation and still has poor MSE and QLIKE. That is RV-OLS expanding: it has the highest correlation (0.43) and the worst losses, because it forecasts 9.8 on average when realized variance averages 5.4, and falls short of RV on only 12% of days. GARCH-t is also miscalibrated, 32% too high on average. Its MZ slope of 0.53 says it is also too reactive: when it forecasts high, RV tends to come in lower.

A quiet day does not prove a high forecast wrong. The forecast is the variance of a distribution, not a prediction of one number. If tomorrow's true variance is high, a calm day is still quite likely, especially with $\nu \approx 3$, where most draws of z^2 are small and a few are huge. RV is in turn a noisy reading of the true variance. One observation cannot separate a bad forecast from an ordinary draw. Calibration can only be judged over many days, as in Table 3.

[1] Re-estimating RV-OLS at every origin makes it comparable to GARCH and to Part VI. Frozen at the end-2024 estimates, it does slightly worse: MSE 62.7, QLIKE 2.782.

Significance versus usefulness. Both questions get a different answer. Does today's RV carry significant information about tomorrow's? Yes, with $t = 12.5$. Does the model forecast new days well enough to be useful? Only modestly: the best rule's Mincer-Zarnowitz R^2 is 0.18, so more than four fifths of the daily variation in RV is unforecastable noise. A coefficient can be very precisely different from zero and still leave most of the variation unexplained. Its intercept can also come from a period that no longer applies, as here.

GARCH versus RV-OLS dynamics (Figure 3). The two rules react to different things. RV-OLS is almost a rescaled copy of yesterday's RV (correlation 0.997): it jumps after every intraday storm and forgets it within two days (average daily change 2.3). GARCH only sees the *daily* return. It moves smoothly (average daily change 0.5), reacts to large closing moves and keeps them in memory for weeks. On 20 January 2025, RV hit 60 while the day closed only +0.9%: GARCH did not react at all, and RV-OLS jumped to 23 for a day that came in at 12. On 5-6 February 2026, returns of −15.2% and +11.8% took GARCH from 9 to 40, and a week later it still forecast 21 while RV was back at 5-10. RV-OLS was back at 7 within two days.

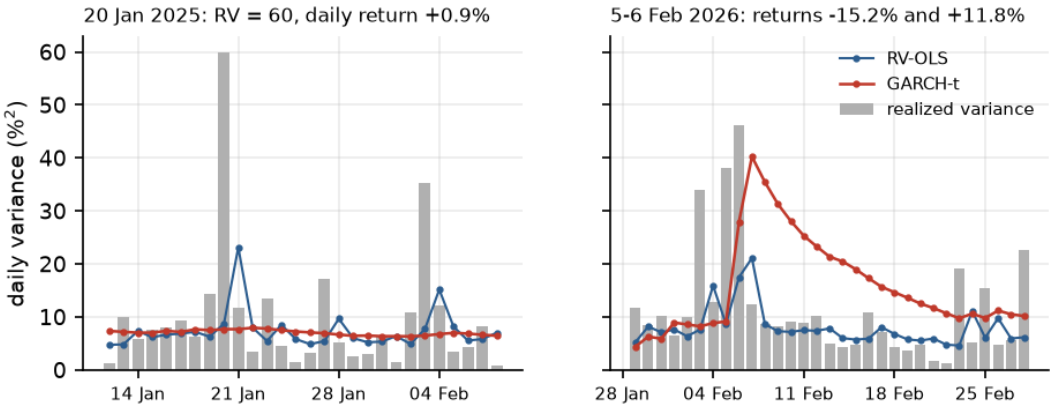


Figure 3: Two episodes, rolling-window forecasts. GARCH misses intraday storms with flat closes and holds onto large closing moves for weeks; RV-OLS reacts to everything for a day.

GARCH is more dynamically flexible in one sense: its memory depends on the state, and it builds up and fades gradually. In 2025-2026, however, volatility spikes were short-lived, so that memory mostly produced over-prediction after shocks. With expanding windows GARCH-t beats RV-OLS on QLIKE (Diebold-Mariano $t = -5.6$, $p < 0.001$). Part VI shows that this advantage comes from calibration, not from the dynamics.

5 Part VI: does distant Bitcoin history still help?

I repeat the exercise with a two-year rolling window: the forecast for 1 January 2025 uses 1 January 2023 to 31 December 2024, and the window moves one day at a time. The evaluation dates do not change.

(a) Rolling versus expanding. For RV-OLS, the two-year window cuts MSE by a third (58.1 to 39.0) and QLIKE from 2.753 to 2.590, both strongly significant (Diebold-Mariano $t \approx -8$). Its average forecast falls from 1.82 to 1.11 times realized variance, while its correlation does not change. The gain is entirely in the level. For GARCH-t the window barely matters (MSE 47.0 vs 48.7, QLIKE 2.615 vs 2.612; neither difference significant). With the rolling window the ranking flips: RV-OLS beats GARCH-t on both losses, although only at the 10% level (MSE $p = 0.07$, QLIKE $p = 0.10$).

(b) The RV persistence coefficient is positive and significant at every one of the 608 origins, but lower than in the full sample. It rises slowly from 0.30 to 0.42, against 0.55 with all history, and the rolling R^2 is 0.09-0.17 rather than 0.29. Today's RV is reliably informative, but less so in the recent, calmer period.

(c) GARCH persistence. With all history, $\alpha + \beta$ sits at 1.000 at every origin. With two years, it is 1.000 until September 2025 and then drifts between 0.95 and 0.98. The 95% interval contains 1 throughout, while α ranges from 0.02 to 0.09 and β moves to compensate. Persistence close to one means shocks never die out: the model has no long-run variance to revert to, and its multi-day forecasts stay flat at today's level. A well-known cause is unmodelled shifts in the level of variance (Lamoureux and Lastrapes, 1990). A GARCH with one long-run level, fitted to a series whose level drifts down, explains the drift as extremely persistent shocks. The first rolling windows include the rise from 2023 to 2024 (mean RV 4.9 to 7.6), and persistence leaves the boundary only once that rise drops out of the window.

(d) What the ranking change says about distant data. Mean RV was 58.6 in 2017, 25.8 in 2018 and has been about 5 since 2023 (Figure 5, appendix). Old observations describe a different level of risk. The rule hurt most is the one that leans hardest on an estimated long-run mean. RV-OLS forgets within a day, so on most days its

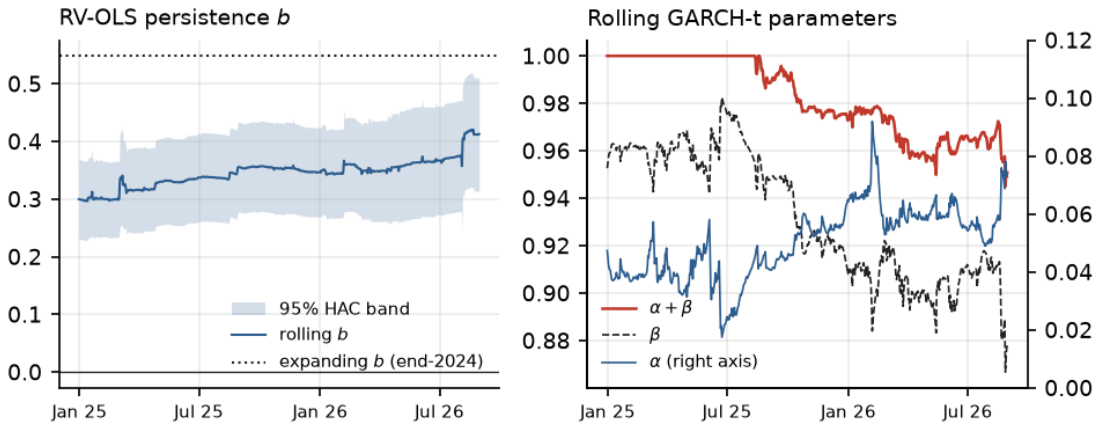


Figure 4: Rolling two-year estimates at each forecast origin. Left: RV-OLS slope with Newey-West 95% band. Right: GARCH-t parameters.

forecast *is* its long-run mean, and with all history that mean is 16 against a realized 5. GARCH-t with $\alpha + \beta \approx 1$ barely uses a long-run mean, so it suffers less from old data. That gave it its expanding-window advantage. Distant history therefore does not help: it mostly contaminates the level.

(e) This does not prove a structural break. The same pattern would come from a gradual decline in volatility, from switching between regimes, or from long memory in volatility. Realized variance is known to have long memory, which is why models that combine daily, weekly and monthly averages of RV work well. I did not test for a break, and the evidence comes from a single 20-month path in which a few spikes decide the MSE. Part of the gap is also not economic. In 2017-2018 the sum of hourly squared returns was 1.4-1.7 times the squared daily returns. Since 2019 the ratio has been about 1. The early RV proxy looks inflated, probably by noisier intraday prices in thinner markets, so old data are partly worse data, not just data from another era.

6 Part VII: recommendation to the risk manager

- **Is there a predictable component?** In risk, yes; in direction, no. Today's volatility reliably signals tomorrow's, but at best about 18% of daily variation in realized variance is forecastable.
- **Is GARCH economically sensible?** Yes: it captures clustering, and the Student-t version ($\nu \approx 3$) is clearly better for Bitcoin. Its persistence estimate at one is a warning, not a feature.
- **Forecast quality against RV.** All rules over-forecast risk. The model ranking depends on the window, but the window effect itself is large, significant and consistent.
- **Significance versus out-of-sample.** Every key parameter is significant, and that told us nothing about which rule would forecast best.
- **Information set.** Rolling two-year windows. The cost is noisier parameters (Figure 4); the benefit is a forecast level that matches the current regime.
- **Uses.** Volatility targeting and position sizing, risk limits, margin buffers, and as the σ input to VaR and Expected Shortfall (with Student-t tails). Not as a precise forecast of any single day.
- **Main uncertainty.** One 20-month test period, dominated by a few spikes. The GARCH-versus-RV ranking is only marginally significant and could reverse in a more persistent volatile period, like 2022, where GARCH's memory would pay off.

Executive recommendation. Tomorrow I would implement the realized-variance regression, re-estimated daily on the last two years. It was the most accurate and best-calibrated rule over 2025-2026, and it is simple enough to audit. GARCH-t on the same window would run alongside as a second opinion after large closing moves, where it reacts more lastingly. I would use the forecast to scale Bitcoin positions to a volatility target, to set daily risk limits and margin buffers, and as the volatility input to VaR and Expected Shortfall with fat tails. Confidence should be moderate on the *level* of risk and low on any single day: four fifths of daily variation is unpredictable, and both models still over-forecast risk on average. I would review the window and the calibration every quarter, because the most important lesson from this exercise is that Bitcoin's risk level changes and old data mislead.

References. Lamoureux, C. and Lastrapes, W. (1990), Persistence in variance, structural change and the GARCH model, *JBES* 8(2). Patton, A. (2011), Volatility forecast comparison using imperfect volatility proxies, *Journal of Econometrics* 160(1).

A Additional figures

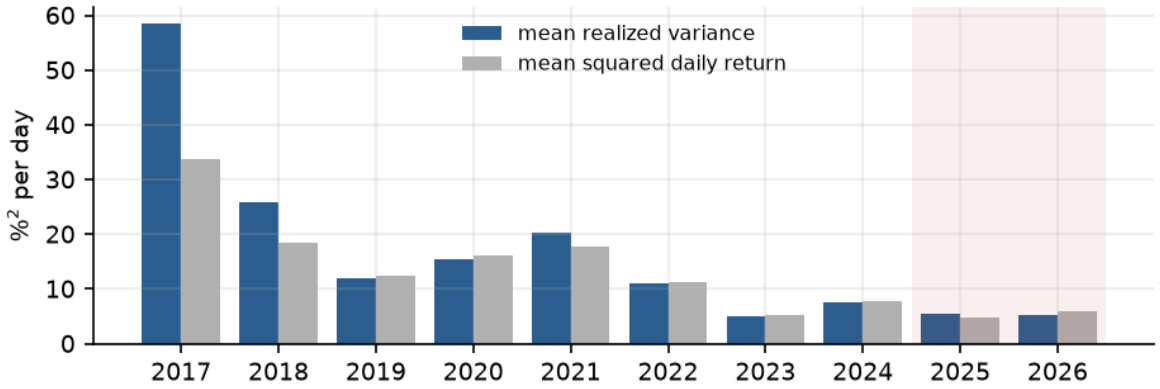


Figure 5: Average daily realized variance and squared daily return by year. The shaded years are the evaluation period. Until 2018, RV is well above the squared daily return; since 2019 the two agree.

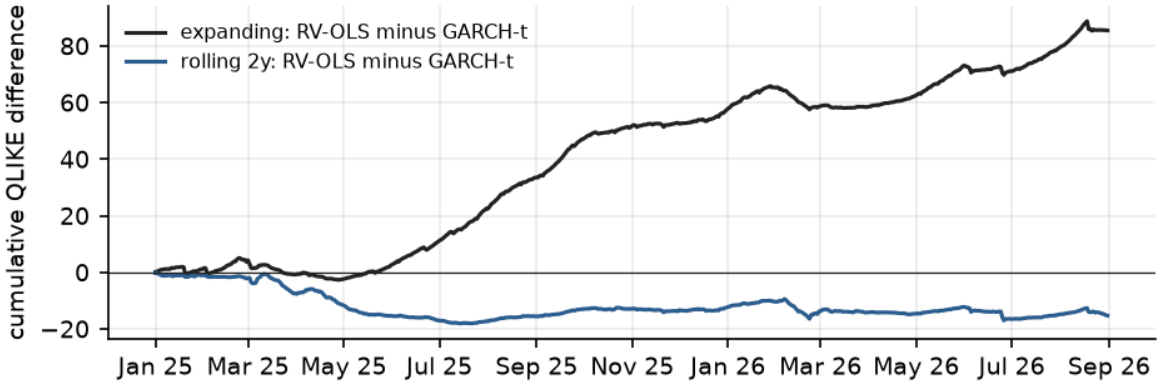


Figure 6: Cumulative QLIKE of RV-OLS minus GARCH-t; rising means GARCH-t is ahead. With expanding windows GARCH pulls ahead steadily from mid-2025. With rolling windows RV-OLS stays slightly ahead.



Figure 7: Bitcoin price and daily returns, 2017-2026. The dashed line marks the start of the evaluation period.

B Python code

The full code is in the repository (`src/`): `task1.py` (Part I), `task2_3.py` (Parts II-III), `forecast.py` (all out-of-sample forecasts), `evaluate.py` (Parts IV-VI) and `figures.py`. Python 3.13 with pandas, statsmodels and `arch`. The core of the exercise, the forecast loop, is reproduced below.

```python
"""One-day-ahead variance forecasts for every day of 2025-01-01 to 2026-08-31.

Four forecasting rules, each re-estimated at every origin t with information up to t only:
  GARCH-t expanding, GARCH-t rolling 2y, RV-OLS expanding, RV-OLS rolling 2y.
Plus RV-OLS frozen at end-2024 (the literal "training information" reading of Task 5).
Writes output/forecasts.parquet (one row per target day) and output/rolling_params.parquet (one row per origin).
"""

import time

import numpy as np
import pandas as pd
import statsmodels.api as sm
from arch import arch_model

from data import FORECASTS, HAC_LAGS, OOS_END, OOS_START, PARAMS, TRAIN_END, load


def garch_t(r: pd.Series, start: np.ndarray | None):
    res = arch_model(r, mean="Constant", vol="GARCH", p=1, q=1, dist="t").fit(
        disp="off", starting_values=start, options={"maxiter": 500})
    if res.convergence_flag != 0: # fall back to arch's own starting values
        res = arch_model(r, mean="Constant", vol="GARCH", p=1, q=1, dist="t").fit(disp="off")
    return res


def rv_pairs(df: pd.DataFrame) -> pd.DataFrame:
    """(RV_s, RV_{s+1}) pairs, kept only when both days have a valid 24-hour RV."""
    p = pd.DataFrame({"x": df["RV"].shift(1), "y": df["RV"],
                      "ok": df["RV_VALID"] & df["RV_VALID"].shift(1, fill_value=False)})
    return p[p["ok"]].dropna()


def rv_ols(pairs: pd.DataFrame):
    return sm.OLS(pairs["y"], sm.add_constant(pairs["x"])).fit()


def main() -> None:
    df = load()
    pairs = rv_pairs(df)
    targets = df.loc[OOS_START:OOS_END].index
    frozen = rv_ols(pairs.loc[:TRAIN_END])

    fc, par = [], []
    start = {"exp": None, "roll": None}
    t0 = time.time()
    for i, day in enumerate(targets):
        origin = day - pd.Timedelta("1D") # information set ends at t
        win_start = origin - pd.DateOffset(years=2) + pd.Timedelta("1D") # 1 Jan 2023 for the first origin
        rv_t = df.at[origin, "RV"]
        row = {"date": day, "RV": df.at[day, "RV"], "RV_VALID": df.at[day, "RV_VALID"], "r": df.at[day, "r"],
               "RV_origin": rv_t, "RV-OLS frozen": frozen.params["const"] + frozen.params["x"] * rv_t}
        prow = {"origin": origin}
        for key, sample in {"exp": df.loc[:origin], "roll": df.loc[win_start:origin]}.items():
            label = "expanding" if key == "exp" else "rolling 2y"
            g = garch_t(sample["r"], start[key])
            start[key] = g.params.values
            row[f"GARCH-t {label}"] = g.forecast(horizon=1, reindex=False).variance.iloc[-1, 0]

            ols = rv_ols(pairs.loc[sample.index[1]:origin])
            row[f"RV-OLS {label}"] = ols.params["const"] + ols.params["x"] * rv_t

            hac = ols.get_robustcov_results(cov_type="HAC", maxlags=HAC_LAGS)
            b, se = hac.params[1], hac.bse[1]
            cov = g.param_cov
            s = g.params["alpha[1]"] + g.params["beta[1]"]
            s_se = (cov.loc["alpha[1]", "alpha[1]"] + cov.loc["beta[1]", "beta[1]"]
                    + 2 * cov.loc["alpha[1]", "beta[1]"]) ** 0.5
            prow |= {f"b {label}": b, f"b lo {label}": b - 1.96 * se, f"b hi {label}": b + 1.96 * se,
                     f"R2 {label}": ols.rsquared, f"n pairs {label}": int(ols.nobs),
                     f"alpha {label}": g.params["alpha[1]"], f"beta {label}": g.params["beta[1]"],
                     f"a+b {label}": s, f"a+b lo {label}": s - 1.96 * s_se, f"a+b hi {label}": s + 1.96 * s_se,
                     f"nu {label}": g.params["nu"], f"n {label}": len(sample)}
        fc.append(row)
        par.append(prow)
        if i % 50 == 0:
            print(f"{day.date()} {i + 1}/{len(targets)} {time.time() - t0:.0f}s", flush=True)

    pd.DataFrame(fc).set_index("date").to_parquet(FORECASTS)
    pd.DataFrame(par).set_index("origin").to_parquet(PARAMS)
    print(f"done: {len(fc)} forecast days in {time.time() - t0:.0f}s")
```

```
if __name__ == "__main__":
    main()
```